DATA ANALYST - FOREX TRADING SIGNAL AI AGENT

Category: Quantitative Finance | FX Markets | Reinforcement Learning | DAX Analytics

Timeline: 15 Days | AI-Accelerated Innovation & Research Project

Target: Finance / Engineering Students & FX Analysts

TASK:

Build an AI Agent that generates Forex trading signals using reinforcement learning, technical analysis, and machine learning models, with comprehensive Power BI dashboards driven by advanced DAX measures for real-time signal monitoring, risk management, and performance attribution across multiple currency pairs.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

This project positions students as Lead FX Quantitative Strategists at a fictional proprietary trading firm – Meridian FX Capital, headquartered in Mumbai with desks in Singapore and London. Meridian manages a $500 million FX book trading G10 and Asian EM currency pairs, and the Chief Trading Officer has tasked the team with building an AI-driven signal generation platform.

The core challenge: Global central bank policy divergence has created unprecedented volatility across FX markets. The Federal Reserve, ECB, Bank of Japan, and RBI are on different rate trajectories, generating complex carry trade dynamics and trend reversals. Manual signal generation can no longer keep pace with market microstructure changes that occur in milliseconds.

Students will design FX Signal Lab – a gamified simulation platform that trains FX traders through realistic multi-currency signal generation scenarios. The platform combines reinforcement learning agents with traditional technical analysis, deploying advanced Power BI dashboards powered by over 120 DAX measures for real-time signal monitoring, risk decomposition, and performance attribution.

The global FX market trades over $7.5 trillion daily (BIS 2022 Triennial Survey), making it the largest and most liquid financial market. India's FX turnover exceeds $35 billion daily, with USD/INR being the most actively traded pair. This project bridges the gap between academic FX theory and the quantitative signal generation workflows used by institutional FX desks at major banks and hedge funds.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART A: TRAINING MATERIAL & LEARNING RESOURCES

SECTION A1: DOMAIN FUNDAMENTALS – THE FOREX MARKET ECOSYSTEM

A1.1 Introduction To Foreign Exchange Markets

The foreign exchange market is a decentralised global marketplace where currencies are traded. Unlike equity markets with centralized exchanges, FX operates through an over-the-counter (OTC) network of banks, brokers, institutional investors, and retail traders across all time zones. Understanding market microstructure is fundamental to building effective trading signals.

Key FX Market Characteristics:

- 24-hour market operating across Sydney, Tokyo, London, and New York sessions with overlapping liquidity windows
- Spot, forward, swap, and options segments – spot accounts for approximately 28% of total turnover
- Currency pairs quoted as Base/Quote (e.g., EUR/USD = euros per US dollar); pip = 0.0001 for most pairs
- Bid-ask spread is the primary transaction cost; tighter for G10 majors (0.5-2 pips), wider for EM pairs (5-50 pips)
- Leverage commonly 50:1 to 100:1 in institutional FX; RBI limits retail leverage in India

Currency Pair Classification:

Category	Pairs	Avg Daily Volume	Typical Spread
G10 Majors	EUR/USD, USD/JPY, GBP/USD, USD/CHF	$2.1T combined	0.5-1.5 pips
G10 Crosses	EUR/GBP, EUR/JPY, GBP/JPY, AUD/NZD	$800B combined	1.5-3.0 pips
Asian EM	USD/INR, USD/CNH, USD/SGD, USD/THB	$350B combined	3-15 pips
LatAm EM	USD/BRL, USD/MXN, USD/CLP	$120B combined	5-25 pips
Commodity FX	AUD/USD, NZD/USD, USD/CAD, USD/NOK	$500B combined	1-3 pips

A1.2 Exchange Rate Determination Theories

Purchasing Power Parity (Ppp):

The fundamental long-run equilibrium model: S(t) = P_domestic / P_foreign, where S is the spot exchange rate and P represents the price level. Relative PPP states that the rate of depreciation of a currency is approximately equal to the difference in inflation rates.

```
Relative PPP: ΔS/S ≈ π_domestic − π_foreign
Example: If India inflation = 6%, US inflation = 3%
Expected INR depreciation ≈ 3% annually
```

Interest Rate Parity (Irp):

Covered Interest Rate Parity (CIP) links spot and forward rates to interest rate differentials. This is a no-arbitrage condition that must hold in efficient markets:

```
Forward Rate / Spot Rate = (1 + r_domestic) / (1 + r_foreign)
Forward Premium = (F - S) / S = (r_d - r_f) / (1 + r_f)
```

Python Implementation – IRP Calculator:

```
def forward_rate(spot, r_domestic, r_foreign, days=90):
    return spot * (1 + r_domestic * days/365) / (1 + r_foreign *
days/365)

def carry_trade_return(spot_entry, spot_exit, r_high, r_low,
days=365):
    interest_diff = (r_high - r_low) * days / 365
    fx_return = (spot_exit - spot_entry) / spot_entry
    return interest_diff + fx_return

# Example: USD/INR carry trade
spot = 83.50
fwd_3m = forward_rate(83.50, 0.065, 0.0525, 90)
print(f'3M Forward: {fwd_3m:.4f}')  # ~83.76
```

Balance Of Payments Approach:

Current account deficits create currency depreciation pressure as more domestic currency is sold to purchase imports. Capital account surpluses (FDI/FPI inflows) provide offsetting appreciation pressure. India's current account deficit of 1-2% of GDP is offset by strong capital inflows, keeping USD/INR relatively stable.

A1.3 Technical Analysis For Fx Signal Generation

Moving Average Systems:

Moving averages are the foundation of trend-following FX strategies. The most common system uses dual moving average crossovers to generate buy/sell signals.

Python Implementation – Complete MA System:

```
import pandas as pd
import numpy as np

class MovingAverageSignal:
    def __init__(self, short_window=10, long_window=50,
ema=True):
        self.short = short_window
        self.long = long_window
        self.use_ema = ema

    def generate_signals(self, df):
        if self.use_ema:
            df['SMA_Short'] =
df['Close'].ewm(span=self.short).mean()
            df['SMA_Long'] =
df['Close'].ewm(span=self.long).mean()
        else:
            df['SMA_Short'] =
df['Close'].rolling(self.short).mean()
            df['SMA_Long'] =
df['Close'].rolling(self.long).mean()
        df['Signal'] = 0
        df.loc[df['SMA_Short'] > df['SMA_Long'], 'Signal'] = 1
# Buy
        df.loc[df['SMA_Short'] < df['SMA_Long'], 'Signal'] = -1
# Sell
        df['Position'] = df['Signal'].diff()
        return df

    def backtest(self, df, initial_capital=100000,
lot_size=100000):
        df = self.generate_signals(df)
        capital = initial_capital
        position = 0
        trades = []
        for i, row in df.iterrows():
            if row['Position'] == 2:  # Buy signal
                position = lot_size
                entry = row['Close']
            elif row['Position'] == -2 and position > 0:  # Sell
signal
                pnl = (row['Close'] - entry) * position
                capital += pnl
                trades.append({'entry': entry, 'exit':
row['Close'], 'pnl': pnl})
```

```
                position = 0
        return pd.DataFrame(trades), capital
```

Relative Strength Index (RSI):

RSI measures the speed and magnitude of price movements on a 0-100 scale. Values above 70 indicate overbought conditions; below 30 indicates oversold. In FX, RSI divergence (price making new highs while RSI makes lower highs) is a powerful reversal signal.

```
def compute_rsi(series, period=14):
    delta = series.diff()
    gain = delta.where(delta > 0, 0).rolling(period).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(period).mean()
    rs = gain / loss
    return 100 - (100 / (1 + rs))
```

Bollinger Bands:

```
def bollinger_bands(series, period=20, num_std=2):
    sma = series.rolling(period).mean()
    std = series.rolling(period).std()
    upper = sma + num_std * std
    lower = sma - num_std * std
    pct_b = (series - lower) / (upper - lower)
    bandwidth = (upper - lower) / sma
    return upper, sma, lower, pct_b, bandwidth
```

MACD (Moving Average Convergence Divergence):

```
def compute_macd(series, fast=12, slow=26, signal=9):
    ema_fast = series.ewm(span=fast).mean()
    ema_slow = series.ewm(span=slow).mean()
    macd_line = ema_fast - ema_slow
    signal_line = macd_line.ewm(span=signal).mean()
    histogram = macd_line - signal_line
    return macd_line, signal_line, histogram
```

Ichimoku Cloud:

```
def ichimoku(df, tenkan=9, kijun=26, senkou_b=52):
    high_t = df['High'].rolling(tenkan).max()
    low_t = df['Low'].rolling(tenkan).min()
    df['Tenkan'] = (high_t + low_t) / 2
    high_k = df['High'].rolling(kijun).max()
    low_k = df['Low'].rolling(kijun).min()
    df['Kijun'] = (high_k + low_k) / 2
    df['Senkou_A'] = ((df['Tenkan'] + df['Kijun']) /
2).shift(kijun)
    high_s = df['High'].rolling(senkou_b).max()
    low_s = df['Low'].rolling(senkou_b).min()
    df['Senkou_B'] = ((high_s + low_s) / 2).shift(kijun)
    df['Chikou'] = df['Close'].shift(-kijun)
    return df
```

A1.4 Composite Signal Generation

Production FX signal systems combine multiple indicators using a weighted scoring mechanism. Each indicator votes, and signals are generated only when consensus exceeds a threshold.

```
class CompositeSignalEngine:
    def __init__(self):
        self.weights = {
            'ma_cross': 0.25,
            'rsi': 0.20,
            'macd': 0.20,
            'bollinger': 0.15,
            'ichimoku': 0.20
        }

    def score(self, df):
        score = 0
```

```
        if df['SMA_Short'].iloc[-1] > df['SMA_Long'].iloc[-1]:
score += self.weights['ma_cross']

        else: score -= self.weights['ma_cross']

        rsi_val = df['RSI'].iloc[-1]

        if rsi_val < 30: score += self.weights['rsi']

        elif rsi_val > 70: score -= self.weights['rsi']

        if df['MACD_Hist'].iloc[-1] > 0: score +=
self.weights['macd']

        else: score -= self.weights['macd']

        if df['Close'].iloc[-1] < df['BB_Lower'].iloc[-1]: score
+= self.weights['bollinger']

        elif df['Close'].iloc[-1] > df['BB_Upper'].iloc[-1]:
score -= self.weights['bollinger']

        if df['Close'].iloc[-1] > df['Senkou_A'].iloc[-1]: score
+= self.weights['ichimoku']

        else: score -= self.weights['ichimoku']

        return score


    def generate_signal(self, df, threshold=0.4):

        s = self.score(df)

        if s >= threshold: return 'STRONG_BUY'

        elif s >= 0.2: return 'BUY'

        elif s <= -threshold: return 'STRONG_SELL'
        elif s <= -0.2: return 'SELL'

        else: return 'HOLD'
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A2: REINFORCEMENT LEARNING FOR FX TRADING

A2.1 Why Reinforcement Learning For FX

Reinforcement Learning (RL) is uniquely suited for FX trading because the problem naturally maps to a Markov Decision Process: the agent observes market state, takes actions (buy/sell/hold), and receives rewards (trading P&L). Unlike supervised learning which requires labelled data, RL learns optimal policies through interaction with market environments.

Mdp Formulation For Fx Trading:

- State Space S: [Close, Volume, RSI, MACD, BB_%B, Carry_Spread, Position, Unrealised_PnL]
- Action Space A: {Long, Short, Flat} for single-pair; {-1, -0.5, 0, 0.5, 1} for position sizing
- Reward R: Realised P&L adjusted for transaction costs and risk (Sharpe-weighted returns)
- Discount Factor γ: 0.99 (high – FX positions are typically held hours to days)
- Episode: One trading day (intraday) or one month (swing trading)

A2.2 PPO Agent For FX Trading – Complete Implementation

```
import gym
import numpy as np
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv

class ForexTradingEnv(gym.Env):
    def __init__(self, df, window_size=20,
initial_balance=100000):
        super().__init__()
        self.df = df.reset_index(drop=True)
        self.window = window_size
        self.initial_balance = initial_balance
        self.action_space = gym.spaces.Discrete(3)  # 0=Hold,
1=Buy, 2=Sell
        self.observation_space = gym.spaces.Box(
            low=-np.inf, high=np.inf,
            shape=(window_size, 8), dtype=np.float32)
        self.reset()

    def _get_observation(self):
        start = self.current_step - self.window
        end = self.current_step
        obs = self.df.iloc[start:end][

['Close','Volume','RSI','MACD','BB_pctB','SMA_Short','SMA_Long',
'ATR']
```

```
        ].values.astype(np.float32)
        return obs

    def reset(self):
        self.balance = self.initial_balance
        self.position = 0  # -1=short, 0=flat, 1=long
        self.entry_price = 0
        self.current_step = self.window
        self.total_pnl = 0
        self.trades = []
        return self._get_observation()

    def step(self, action):
        current_price = self.df.iloc[self.current_step]['Close']
        reward = 0
        spread_cost = 0.0002  # 2 pips
        if action == 1 and self.position <= 0:  # Buy
            if self.position == -1:
                pnl = (self.entry_price - current_price -
spread_cost) * 100000
                reward = pnl
                self.total_pnl += pnl
            self.position = 1
            self.entry_price = current_price + spread_cost
        elif action == 2 and self.position >= 0:  # Sell
            if self.position == 1:
                pnl = (current_price - self.entry_price -
spread_cost) * 100000
                reward = pnl
                self.total_pnl += pnl
            self.position = -1
            self.entry_price = current_price - spread_cost
        self.current_step += 1
        done = self.current_step >= len(self.df) - 1
        return self._get_observation(), reward, done, {}

# Training
env = DummyVecEnv([lambda: ForexTradingEnv(df_train)])
model = PPO('MlpPolicy', env, verbose=1,
    learning_rate=3e-4, n_steps=2048, batch_size=64,
    n_epochs=10, gamma=0.99, clip_range=0.2)
model.learn(total_timesteps=100000)
```

A2.3 DQN Agent For FX Trading

```
from stable_baselines3 import DQN
model_dqn = DQN('MlpPolicy', env, verbose=1,
    learning_rate=1e-4, buffer_size=50000,
    exploration_fraction=0.2, exploration_final_eps=0.05,
    target_update_interval=1000, batch_size=32)
model_dqn.learn(total_timesteps=100000)
```

A2.4 A2C Agent for Multi-Pair Trading

```
from stable_baselines3 import A2C
model_a2c = A2C('MlpPolicy', env, verbose=1,
    learning_rate=7e-4, n_steps=5, gamma=0.99,
    ent_coef=0.01, vf_coef=0.5)
model_a2c.learn(total_timesteps=100000)
```

A2.5 R Implementation – RL For FX:

```
library(reticulate)
library(ReinforcementLearning)
py_install('stable-baselines3')
data <- read.csv('forex_data.csv')
py_run_string("
from stable_baselines3 import PPO
from custom_env import ForexTradingEnv
env = ForexTradingEnv(df=data, window_size=20)
model = PPO('MlpPolicy', env, verbose=1)
model.learn(total_timesteps=50000)
")
signals <- py_run_string('model.predict(obs)', convert=TRUE)
write.csv(signals, 'rl_trading_signals.csv')
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A3: MACHINE LEARNING FOR SIGNAL CLASSIFICATION

A3.1 Feature Engineering For FX Prediction

Effective ML models for FX require carefully engineered features that capture momentum, volatility, carry, and microstructure dynamics.

```
def engineer_features(df):
    # Momentum features
    for lag in [1, 5, 10, 20, 60]:
        df[f'return_{lag}d'] = df['Close'].pct_change(lag)
    # Volatility features
    for w in [5, 10, 20, 60]:
        df[f'vol_{w}d'] =
df['Close'].pct_change().rolling(w).std() * np.sqrt(252)
    # Technical indicators
    df['RSI_14'] = compute_rsi(df['Close'], 14)
    df['MACD'], df['MACD_Signal'], df['MACD_Hist'] =
compute_macd(df['Close'])
    df['BB_Upper'], df['BB_Mid'], df['BB_Lower'], df['BB_pctB'],
df['BB_BW'] = bollinger_bands(df['Close'])
    # Volume features
    df['Volume_SMA'] = df['Volume'].rolling(20).mean()
    df['Volume_Ratio'] = df['Volume'] / df['Volume_SMA']
    # ATR for position sizing
    high_low = df['High'] - df['Low']
    high_close = abs(df['High'] - df['Close'].shift())
    low_close = abs(df['Low'] - df['Close'].shift())
    tr = pd.concat([high_low, high_close, low_close],
axis=1).max(axis=1)
    df['ATR_14'] = tr.rolling(14).mean()
    # Session indicators
    df['Hour'] = df.index.hour
    df['Is_London'] = ((df['Hour'] >= 8) & (df['Hour'] <=
16)).astype(int)
    df['Is_NY'] = ((df['Hour'] >= 13) & (df['Hour'] <=
21)).astype(int)
    df['Is_Overlap'] = ((df['Hour'] >= 13) & (df['Hour'] <=
16)).astype(int)
    return df.dropna()
```

A3.2 Random Forest Signal Classifier:

```
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import classification_report,
confusion_matrix

# Target: 1 if next-day return > 0, else 0
df['Target'] = (df['Close'].shift(-1) > df['Close']).astype(int)
```

```
features =
['return_1d','return_5d','return_10d','return_20d','vol_5d','vol
_20d',

'RSI_14','MACD_Hist','BB_pctB','BB_BW','Volume_Ratio','ATR_14',
    'Is_London','Is_NY','Is_Overlap']

tscv = TimeSeriesSplit(n_splits=5)
rf = RandomForestClassifier(n_estimators=200, max_depth=8,
min_samples_leaf=20, random_state=42)
for train_idx, test_idx in tscv.split(X):
    rf.fit(X.iloc[train_idx], y.iloc[train_idx])
    preds = rf.predict(X.iloc[test_idx])
    print(classification_report(y.iloc[test_idx], preds))
```

A3.3 XGBoost For Signal Strength Prediction

```
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=300, max_depth=5,
learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1,
reg_lambda=1.0)
xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)],
    early_stopping_rounds=20, verbose=False)
```

A3.4 LSTM Neural Network for Sequential Signals

```
import tensorflow as tf

def build_lstm(input_shape, n_classes=3):
    model = tf.keras.Sequential([
        tf.keras.layers.LSTM(128, return_sequences=True,
input_shape=input_shape),
        tf.keras.layers.Dropout(0.3),
        tf.keras.layers.LSTM(64, return_sequences=False),
        tf.keras.layers.Dropout(0.3),
        tf.keras.layers.Dense(32, activation='relu'),
        tf.keras.layers.Dense(n_classes, activation='softmax')
    ])
    model.compile(optimizer='adam',
loss='categorical_crossentropy',
        metrics=['accuracy'])
    return model

# Reshape for LSTM: (samples, timesteps, features)
X_lstm = np.array([X.values[i-20:i] for i in range(20, len(X))])
model = build_lstm((20, X.shape[1]))
```

```
model.fit(X_lstm, y_cat, epochs=50, batch_size=32,
validation_split=0.2)
```

A3.5 SHAP Explainability for FX Signals

```
import shap
explainer = shap.TreeExplainer(xgb)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test, feature_names=features)
shap.force_plot(explainer.expected_value, shap_values[0],
X_test.iloc[0])
```

A3.6 R Implementation

```
library(tidyverse); library(caret); library(xgboost);
library(randomForest)
rf_model <- randomForest(Target ~ ., data=train_data, ntree=200,
mtry=5)
xgb_model <- xgboost(data=as.matrix(train_x), label=train_y,
    max_depth=5, eta=0.05, nrounds=300,
objective='binary:logistic', verbose=0)
library(keras)
lstm_model <- keras_model_sequential() %>%
    layer_lstm(units=128, return_sequences=TRUE,
input_shape=c(20, ncol(X))) %>%
    layer_dropout(0.3) %>%
    layer_lstm(units=64) %>%
    layer_dropout(0.3) %>%
    layer_dense(units=3, activation='softmax')
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A4: ADVANCED DAX FOR FOREX TRADING DASHBOARDS

This section contains the comprehensive DAX measure library for building production-grade Power BI dashboards for FX signal monitoring. These measures are designed for a star-schema data model with fact tables for trades, signals, and market data, linked to dimension tables for currency pairs, time periods, sessions, and strategies.

Data Model Architecture

Table	Type	Key Fields	Granularity
FactTrades	Fact	TradeID, PairID, DateKey, Signal, EntryPrice, ExitPrice, PnL	One row per trade
FactSignals	Fact	SignalID, PairID, DateTimeKey, SignalType, Confidence, Score	One row per signal
FactMarketData	Fact	PairID, DateTimeKey, Open, High, Low, Close, Volume	One row per candle
DimCurrencyPair	Dimension	PairID, PairName, BaseCurrency, QuoteCurrency, Region	One row per pair
DimDate	Dimension	DateKey, Date, Year, Quarter, Month, DayOfWeek	One row per day
DimTime	Dimension	TimeKey, Hour, Minute, Session, IsOverlap	One row per minute
DimStrategy	Dimension	StrategyID, StrategyName, Type, RiskLevel	One row per strategy

A4.1 Core Signal Measures

```
// Total Signals Generated
   Total Signals = COUNTROWS(FactSignals)

// Buy Signals Count
   Buy Signals = CALCULATE(COUNTROWS(FactSignals),
   FactSignals[SignalType] = "BUY")

// Sell Signals Count
   Sell Signals = CALCULATE(COUNTROWS(FactSignals),
   FactSignals[SignalType] = "SELL")

// Strong Buy Signals
   Strong Buy Signals =
       CALCULATE(
           COUNTROWS(FactSignals),
           FactSignals[SignalType] = "STRONG_BUY",
           FactSignals[Confidence] >= 0.8
       )
```

```
// Signal Hit Rate (Accuracy)
   Signal Hit Rate =
       VAR TotalExecuted = CALCULATE(COUNTROWS(FactSignals),
   FactSignals[Executed] = TRUE())
       VAR Profitable = CALCULATE(COUNTROWS(FactSignals),
   FactSignals[Executed] = TRUE(), FactSignals[PnL] > 0)
       RETURN DIVIDE(Profitable, TotalExecuted, 0)

// Average Signal Confidence
   Avg Signal Confidence = AVERAGEX(FactSignals,
   FactSignals[Confidence])

// Weighted Signal Score
   Weighted Signal Score =
       SUMX(
           FactSignals,
           FactSignals[Score] * FactSignals[Confidence]
       ) / SUMX(FactSignals, FactSignals[Confidence])

// Signal Frequency (Signals Per Day)
   Signal Frequency =
       DIVIDE(
           [Total Signals],
           DISTINCTCOUNT(FactSignals[DateKey]),
           0
       )

// Signal Distribution by Type
   Signal Distribution =
       VAR CurrentType = SELECTEDVALUE(FactSignals[SignalType])
       VAR TypeCount = CALCULATE(COUNTROWS(FactSignals),
   FactSignals[SignalType] = CurrentType)
       RETURN DIVIDE(TypeCount, [Total Signals], 0)
```

A4.2 Trade Performance Measures

```
// Total Trades
   Total Trades = COUNTROWS(FactTrades)

// Winning Trades
   Winning Trades = CALCULATE(COUNTROWS(FactTrades),
   FactTrades[PnL] > 0)

// Losing Trades
   Losing Trades = CALCULATE(COUNTROWS(FactTrades), FactTrades[PnL]
   < 0)

// Win Rate
```

```
   Win Rate = DIVIDE([Winning Trades], [Total Trades], 0)

// Total P&L
   Total PnL = SUMX(FactTrades, FactTrades[PnL])

// Average Win
   Average Win =
       AVERAGEX(
           FILTER(FactTrades, FactTrades[PnL] > 0),
           FactTrades[PnL]
       )

// Average Loss
   Average Loss =
       AVERAGEX(
           FILTER(FactTrades, FactTrades[PnL] < 0),
           FactTrades[PnL]
       )

// Profit Factor
   Profit Factor =
       VAR GrossProfit = SUMX(FILTER(FactTrades, FactTrades[PnL] >
   0), FactTrades[PnL])
       VAR GrossLoss = ABS(SUMX(FILTER(FactTrades, FactTrades[PnL]
   < 0), FactTrades[PnL]))
       RETURN DIVIDE(GrossProfit, GrossLoss, 0)

// Expectancy (Average Profit Per Trade)
   Expectancy =
       VAR WR = [Win Rate]
       VAR AvgW = [Average Win]
       VAR AvgL = ABS([Average Loss])
       RETURN WR * AvgW - (1 - WR) * AvgL

// Maximum Consecutive Wins
   Max Consecutive Wins =
       VAR TradesTable =
           ADDCOLUMNS(
               SELECTCOLUMNS(FactTrades, "TradeDate",
   FactTrades[TradeDate], "IsWin", IF(FactTrades[PnL]>0, 1, 0)),
               "RowNum", RANKX(ALL(FactTrades),
   FactTrades[TradeDate], , ASC, Dense)
           )
       VAR Groups =
           ADDCOLUMNS(TradesTable,
```

```
            "GroupID", [RowNum] - RANKX(FILTER(TradesTable,
[IsWin]=1), [TradeDate], , ASC, Dense)
        )
    RETURN MAXX(FILTER(Groups, [IsWin]=1),
COUNTROWS(FILTER(Groups, [GroupID]=[GroupID] && [IsWin]=1)))

// Payoff Ratio
Payoff Ratio = DIVIDE([Average Win], ABS([Average Loss]), 0)

// Calmar Ratio (Annualised Return / Max Drawdown)
Calmar Ratio =
    VAR AnnReturn = [Annualised Return]
    VAR MaxDD = ABS([Maximum Drawdown Pct])
    RETURN DIVIDE(AnnReturn, MaxDD, 0)
```

A4.3 Risk Management DAX Measures

```
// Daily P&L
Daily PnL =
    SUMX(
        FILTER(FactTrades, FactTrades[DateKey] =
SELECTEDVALUE(DimDate[DateKey])),
        FactTrades[PnL]
    )

// Cumulative P&L
Cumulative PnL =
    CALCULATE(
        [Total PnL],
        FILTER(
            ALL(DimDate),
            DimDate[Date] <= MAX(DimDate[Date])
        )
    )

// Running Maximum P&L (for drawdown calculation)
Running Max PnL =
    MAXX(
        FILTER(
            ALL(DimDate),
            DimDate[Date] <= MAX(DimDate[Date])
        ),
        [Cumulative PnL]
    )

// Current Drawdown
Current Drawdown = [Cumulative PnL] - [Running Max PnL]
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

```
// Maximum Drawdown
   Maximum Drawdown =
       MINX(
           ALL(DimDate),
           VAR CumPnL = CALCULATE([Cumulative PnL], DimDate[Date]
   <= DimDate[Date])
           VAR RunMax = CALCULATE([Running Max PnL], DimDate[Date]
   <= DimDate[Date])
           RETURN CumPnL - RunMax
       )

// Maximum Drawdown Percentage
   Maximum Drawdown Pct =
       VAR InitialCapital = 100000
       RETURN DIVIDE([Maximum Drawdown], InitialCapital + [Running
   Max PnL], 0)

// Value at Risk (Parametric 95%)
   VaR 95% =
       VAR DailyReturns =
           SELECTCOLUMNS(
               SUMMARIZE(FactTrades, DimDate[Date], "DailyPnL",
   [Daily PnL]),
               "DPnL", [DailyPnL]
           )
       VAR AvgReturn = AVERAGEX(DailyReturns, [DPnL])
       VAR StdDev =
           SQRT(
               AVERAGEX(DailyReturns, ([DPnL] - AvgReturn)^2)
           )
       RETURN AvgReturn - 1.645 * StdDev

// Conditional VaR (Expected Shortfall 95%)
   CVaR 95% =
       VAR VaR95 = [VaR 95%]
       VAR TailLosses =
           FILTER(
               SUMMARIZE(FactTrades, DimDate[Date], "DailyPnL",
   [Daily PnL]),
               [DailyPnL] <= VaR95
           )
       RETURN AVERAGEX(TailLosses, [DailyPnL])

// Sharpe Ratio (Annualised)
   Sharpe Ratio =
```

```
        VAR DailyReturns =
            SUMMARIZE(FactTrades, DimDate[Date], "DailyPnL", [Daily
    PnL])
        VAR AvgDaily = AVERAGEX(DailyReturns, [DailyPnL])
        VAR StdDaily =
            SQRT(AVERAGEX(DailyReturns, ([DailyPnL] - AvgDaily)^2))
        VAR RiskFreeDaily = 0.05 / 252
        RETURN DIVIDE(AvgDaily - RiskFreeDaily, StdDaily, 0) *
    SQRT(252)

// Sortino Ratio (Downside Risk Only)
    Sortino Ratio =
        VAR DailyReturns =
            SUMMARIZE(FactTrades, DimDate[Date], "DailyPnL", [Daily
    PnL])
        VAR AvgDaily = AVERAGEX(DailyReturns, [DailyPnL])
        VAR DownsideDev =
            SQRT(AVERAGEX(FILTER(DailyReturns, [DailyPnL] < 0),
    [DailyPnL]^2))
        RETURN DIVIDE(AvgDaily * 252, DownsideDev * SQRT(252), 0)

// Annualised Return
    Annualised Return =
        VAR TotalDays = DISTINCTCOUNT(FactTrades[DateKey])
        VAR TotalRet = DIVIDE([Total PnL], 100000, 0)
        RETURN (1 + TotalRet) ^ (252 / TotalDays) - 1

// Annualised Volatility
    Annualised Volatility =
        VAR DailyReturns =
            SUMMARIZE(FactTrades, DimDate[Date], "DailyPnL", [Daily
    PnL])
        VAR AvgDaily = AVERAGEX(DailyReturns, [DailyPnL])
        RETURN SQRT(AVERAGEX(DailyReturns, ([DailyPnL] -
    AvgDaily)^2)) * SQRT(252)

// Position Sizing (Kelly Criterion)
    Kelly Fraction =
        VAR WR = [Win Rate]
        VAR PR = [Payoff Ratio]
        RETURN WR - (1 - WR) / PR

// Risk-Adjusted Kelly (Half Kelly)
    Half Kelly = [Kelly Fraction] / 2
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A4.4 Currency Pair Analysis DAX Measures

```
// P&L by Currency Pair
   PnL by Pair =
       SUMX(
           FILTER(FactTrades, FactTrades[PairID] =
   SELECTEDVALUE(DimCurrencyPair[PairID])),
           FactTrades[PnL]
       )

// Win Rate by Pair
   Win Rate by Pair =
       VAR CurrentPair = SELECTEDVALUE(DimCurrencyPair[PairID])
       VAR TotalPairTrades = CALCULATE(COUNTROWS(FactTrades),
   FactTrades[PairID] = CurrentPair)
       VAR WinPairTrades = CALCULATE(COUNTROWS(FactTrades),
   FactTrades[PairID] = CurrentPair, FactTrades[PnL] > 0)
       RETURN DIVIDE(WinPairTrades, TotalPairTrades, 0)

// Correlation Matrix (Pair Returns)
   Pair Correlation =
       VAR Pair1 = SELECTEDVALUE(DimCurrencyPair[PairName])
       VAR Pair2 = SELECTEDVALUE(DimCorrelation[PairName2])
       VAR Returns1 =
           SELECTCOLUMNS(
               FILTER(FactMarketData,
   RELATED(DimCurrencyPair[PairName]) = Pair1),
               "Date", FactMarketData[DateTimeKey],
               "Ret1", DIVIDE(FactMarketData[Close] -
   FactMarketData[PrevClose], FactMarketData[PrevClose])
           )
       VAR Returns2 =
           SELECTCOLUMNS(
               FILTER(FactMarketData,
   RELATED(DimCurrencyPair[PairName]) = Pair2),
               "Date", FactMarketData[DateTimeKey],
               "Ret2", DIVIDE(FactMarketData[Close] -
   FactMarketData[PrevClose], FactMarketData[PrevClose])
           )
       VAR JoinedReturns = NATURALLEFTOUTERJOIN(Returns1, Returns2)
       VAR AvgR1 = AVERAGEX(JoinedReturns, [Ret1])
       VAR AvgR2 = AVERAGEX(JoinedReturns, [Ret2])
       VAR Covariance = AVERAGEX(JoinedReturns, ([Ret1] - AvgR1) *
   ([Ret2] - AvgR2))
       VAR Std1 = SQRT(AVERAGEX(JoinedReturns, ([Ret1] - AvgR1) ^
   2))
       VAR Std2 = SQRT(AVERAGEX(JoinedReturns, ([Ret2] - AvgR2) ^
   2))
```

```
        RETURN DIVIDE(Covariance, Std1 * Std2, 0)

// Best Performing Pair (Dynamic)
   Best Pair =
        VAR PairPnL =
            ADDCOLUMNS(
                VALUES(DimCurrencyPair[PairName]),
                "PairTotal", [PnL by Pair]
            )
        RETURN MAXX(PairPnL, [PairTotal])

// Pair Volatility (Rolling 20-day)
   Pair Volatility 20D =
        VAR CurrentPair = SELECTEDVALUE(DimCurrencyPair[PairID])
        VAR Last20 =
            TOPN(20,
                FILTER(FactMarketData, FactMarketData[PairID] =
   CurrentPair),
                FactMarketData[DateTimeKey], DESC
            )
        VAR AvgRet = AVERAGEX(Last20, FactMarketData[DailyReturn])
        RETURN SQRT(AVERAGEX(Last20, (FactMarketData[DailyReturn] -
   AvgRet)^2)) * SQRT(252)
```

A4.5 Session Analysis DAX Measures

```
// P&L by Trading Session
   Session PnL =
        SUMX(
            FILTER(FactTrades, RELATED(DimTime[Session]) =
   SELECTEDVALUE(DimTime[Session])),
            FactTrades[PnL]
        )

// Win Rate by Session
   Session Win Rate =
        VAR SessionTrades =
            FILTER(FactTrades, RELATED(DimTime[Session]) =
   SELECTEDVALUE(DimTime[Session]))
        VAR TotalSess = COUNTROWS(SessionTrades)
        VAR WinSess = COUNTROWS(FILTER(SessionTrades,
   FactTrades[PnL] > 0))
        RETURN DIVIDE(WinSess, TotalSess, 0)

// London-NY Overlap Premium
```

```
    Overlap Premium =
        VAR OverlapPnL = CALCULATE([Total PnL], DimTime[IsOverlap] =
    1)
        VAR NonOverlapPnL = CALCULATE([Total PnL],
    DimTime[IsOverlap] = 0)
        VAR OverlapTrades = CALCULATE([Total Trades],
    DimTime[IsOverlap] = 1)
        VAR NonOverlapTrades = CALCULATE([Total Trades],
    DimTime[IsOverlap] = 0)
        RETURN DIVIDE(OverlapPnL, OverlapTrades) -
    DIVIDE(NonOverlapPnL, NonOverlapTrades)

// Hourly Heatmap Score
    Hourly Score =
        VAR HourlyPnL =
            CALCULATE([Total PnL], DimTime[Hour] =
    SELECTEDVALUE(DimTime[Hour]))
        VAR HourlyTrades =
            CALCULATE([Total Trades], DimTime[Hour] =
    SELECTEDVALUE(DimTime[Hour]))
        RETURN DIVIDE(HourlyPnL, HourlyTrades, 0)
```

A4.6 Technological Indicator DAX Measures

```
// Simple Moving Average (N-Period)
    SMA_N =
        VAR N = SELECTEDVALUE(DimParameter[MA_Period], 20)
        VAR CurrentDate = MAX(FactMarketData[DateTimeKey])
        VAR LastN =
            TOPN(N,
                FILTER(ALL(FactMarketData),
    FactMarketData[DateTimeKey] <= CurrentDate),
                  FactMarketData[DateTimeKey], DESC
            )
        RETURN AVERAGEX(LastN, FactMarketData[Close])

// Exponential Moving Average
    EMA_N =
        VAR N = SELECTEDVALUE(DimParameter[MA_Period], 20)
        VAR Alpha = 2 / (N + 1)
        VAR CurrentDate = MAX(FactMarketData[DateTimeKey])
        VAR AllData =
            FILTER(ALL(FactMarketData), FactMarketData[DateTimeKey]
    <= CurrentDate)
        VAR Ranked =
            ADDCOLUMNS(AllData, "RN", RANKX(AllData,
    FactMarketData[DateTimeKey], , DESC, Dense))
        // Note: True EMA requires iterative calculation;
    approximated via SUMX with decay weights
```

```
        RETURN
            SUMX(
                TOPN(N * 3, Ranked, [RN], ASC),
                FactMarketData[Close] * Alpha * (1 - Alpha) ^ ([RN]
   - 1)
            ) / SUMX(
                TOPN(N * 3, Ranked, [RN], ASC),
                Alpha * (1 - Alpha) ^ ([RN] - 1)
            )

// RSI in DAX
   RSI_14 =
        VAR Period = 14
        VAR CurrentDate = MAX(FactMarketData[DateTimeKey])
        VAR RecentData =
            TOPN(Period + 1,
                FILTER(ALL(FactMarketData),
   FactMarketData[DateTimeKey] <= CurrentDate),
                FactMarketData[DateTimeKey], DESC
            )
        VAR Changes =
            ADDCOLUMNS(RecentData,
                "Change", FactMarketData[Close] -
   FactMarketData[PrevClose]
            )
        VAR AvgGain = AVERAGEX(FILTER(Changes, [Change] > 0),
   [Change])
        VAR AvgLoss = AVERAGEX(FILTER(Changes, [Change] < 0),
   ABS([Change]))
        VAR RS = DIVIDE(AvgGain, AvgLoss, 0)
        RETURN 100 - DIVIDE(100, 1 + RS, 50)

// Bollinger Band Position (%B)
   BB Percent B =
        VAR Period = 20
        VAR NumStd = 2
        VAR SMA = [SMA_N]
        VAR CurrentDate = MAX(FactMarketData[DateTimeKey])
        VAR RecentData =
            TOPN(Period,
                FILTER(ALL(FactMarketData),
   FactMarketData[DateTimeKey] <= CurrentDate),
                FactMarketData[DateTimeKey], DESC
            )
        VAR StdDev =
            SQRT(AVERAGEX(RecentData, (FactMarketData[Close] - SMA)
   ^ 2))
```

```
        VAR Upper = SMA + NumStd * StdDev
        VAR Lower = SMA - NumStd * StdDev
        VAR CurrentPrice = MAXX(RecentData, FactMarketData[Close])
        RETURN DIVIDE(CurrentPrice - Lower, Upper - Lower, 0.5)

// ATR (Average True Range)
   ATR_14 =
        VAR Period = 14
        VAR CurrentDate = MAX(FactMarketData[DateTimeKey])
        VAR RecentData =
            TOPN(Period,
                FILTER(ALL(FactMarketData),
   FactMarketData[DateTimeKey] <= CurrentDate),
                FactMarketData[DateTimeKey], DESC
            )
        RETURN
            AVERAGEX(RecentData,
                MAX(
                    FactMarketData[High] - FactMarketData[Low],
                    MAX(
                        ABS(FactMarketData[High] -
   FactMarketData[PrevClose]),
                        ABS(FactMarketData[Low] -
   FactMarketData[PrevClose])
                    )
                )
            )

// MACD Histogram in DAX
   MACD Histogram =
        VAR EMA12 =
            CALCULATE([EMA_N], DimParameter[MA_Period] = 12)
        VAR EMA26 =
            CALCULATE([EMA_N], DimParameter[MA_Period] = 26)
        VAR MACDLine = EMA12 - EMA26
        // Signal line would require a 9-period EMA of MACD Line
   (computed via calculated column)
        VAR SignalLine = [MACD Signal Line]
        RETURN MACDLine - SignalLine
```

A4.7 Strategy Comparison DAX Measures

```
// Strategy PnL
   Strategy PnL =
       SUMX(
           FILTER(FactTrades, FactTrades[StrategyID] =
   SELECTEDVALUE(DimStrategy[StrategyID])),
           FactTrades[PnL]
       )

// Strategy Win Rate
   Strategy Win Rate =
       VAR StratTrades =
           FILTER(FactTrades, FactTrades[StrategyID] =
   SELECTEDVALUE(DimStrategy[StrategyID]))
       RETURN DIVIDE(
           COUNTROWS(FILTER(StratTrades, FactTrades[PnL] > 0)),
           COUNTROWS(StratTrades), 0)

// Strategy Sharpe Ratio
   Strategy Sharpe =
       VAR StratID = SELECTEDVALUE(DimStrategy[StrategyID])
       VAR DailyRet =
           SUMMARIZE(
               FILTER(FactTrades, FactTrades[StrategyID] =
   StratID),
               DimDate[Date], "DPnL", SUM(FactTrades[PnL])
           )
       VAR AvgD = AVERAGEX(DailyRet, [DPnL])
       VAR StdD = SQRT(AVERAGEX(DailyRet, ([DPnL] - AvgD)^2))
       RETURN DIVIDE(AvgD, StdD, 0) * SQRT(252)

// Strategy Ranking
   Strategy Rank =
       RANKX(
           ALL(DimStrategy),
           [Strategy Sharpe],
           , DESC, Dense
       )

// Strategy Alpha (vs Benchmark)
   Strategy Alpha =
       VAR StratReturn = [Annualised Return]
       VAR BenchReturn =
           CALCULATE([Annualised Return], DimStrategy[StrategyName]
   = "Benchmark_BuyHold")
       RETURN StratReturn - BenchReturn
```

```
// Information Ratio
   Information Ratio =
       VAR Alpha = [Strategy Alpha]
       VAR TrackingError =
           VAR DailyAlpha =
               SUMMARIZE(FactTrades, DimDate[Date],
                   "StratPnL", CALCULATE(SUM(FactTrades[PnL]),
   FactTrades[StrategyID] =
   SELECTEDVALUE(DimStrategy[StrategyID])),
                   "BenchPnL", CALCULATE(SUM(FactTrades[PnL]),
   DimStrategy[StrategyName] = "Benchmark_BuyHold")
               )
           VAR AvgAlpha = AVERAGEX(DailyAlpha, [StratPnL] -
   [BenchPnL])
           RETURN SQRT(AVERAGEX(DailyAlpha, ([StratPnL] -
   [BenchPnL] - AvgAlpha)^2)) * SQRT(252)
       RETURN DIVIDE(Alpha, TrackingError, 0)
```

A4.8 Carry Trade Analysis DAX Measures

```
// Carry Spread
   Carry Spread =
       VAR BaseCcy = SELECTEDVALUE(DimCurrencyPair[BaseCurrency])
       VAR QuoteCcy = SELECTEDVALUE(DimCurrencyPair[QuoteCurrency])
       VAR BaseRate = LOOKUPVALUE(DimInterestRates[Rate],
   DimInterestRates[Currency], BaseCcy)
       VAR QuoteRate = LOOKUPVALUE(DimInterestRates[Rate],
   DimInterestRates[Currency], QuoteCcy)
       RETURN BaseRate - QuoteRate

// Carry P&L Component
   Carry PnL =
       SUMX(
           FactTrades,
           FactTrades[HoldingDays] * [Carry Spread] / 365 *
   FactTrades[PositionSize]
       )

// Spot P&L Component
   Spot PnL = [Total PnL] - [Carry PnL]

// Carry-to-Risk Ratio
   Carry to Risk =
       DIVIDE(ABS([Carry Spread]), [Pair Volatility 20D], 0)
```

A4.9 Monte Carlo Simulation DAX Measures

```
// MC Scenario P&L
   MC Scenario PnL =
       SUMX(
           FactMCScenarios,
           FactMCScenarios[PortfolioPnL]
       )

// MC VaR 95%
   MC VaR 95% = PERCENTILEX.INC(FactMCScenarios,
   FactMCScenarios[PortfolioPnL], 0.05)

// MC VaR 99%
   MC VaR 99% = PERCENTILEX.INC(FactMCScenarios,
   FactMCScenarios[PortfolioPnL], 0.01)

// MC CVaR 95%
   MC CVaR 95% =
       AVERAGEX(
           FILTER(FactMCScenarios, FactMCScenarios[PortfolioPnL] <=
   [MC VaR 95%]),
           FactMCScenarios[PortfolioPnL]
       )

// MC Max Scenario Loss
   MC Worst Case = MINX(FactMCScenarios,
   FactMCScenarios[PortfolioPnL])

// MC Best Scenario Gain
   MC Best Case = MAXX(FactMCScenarios,
   FactMCScenarios[PortfolioPnL])

// MC Mean Expected P&L
   MC Expected PnL = AVERAGEX(FactMCScenarios,
   FactMCScenarios[PortfolioPnL])

// MC Probability of Loss
   MC Loss Probability =
       DIVIDE(
           COUNTROWS(FILTER(FactMCScenarios,
   FactMCScenarios[PortfolioPnL] < 0)),
           COUNTROWS(FactMCScenarios),
           0
       )

// MC Skewness
```

```
   MC Skewness =
       VAR Avg = [MC Expected PnL]
       VAR Std = SQRT(AVERAGEX(FactMCScenarios,
   (FactMCScenarios[PortfolioPnL] - Avg) ^ 2))
       VAR N = COUNTROWS(FactMCScenarios)
       RETURN
           (N / ((N-1)*(N-2))) *
           SUMX(FactMCScenarios, ((FactMCScenarios[PortfolioPnL] -
   Avg) / Std) ^ 3)

// MC Kurtosis
   MC Kurtosis =
       VAR Avg = [MC Expected PnL]
       VAR Std = SQRT(AVERAGEX(FactMCScenarios,
   (FactMCScenarios[PortfolioPnL] - Avg) ^ 2))
       VAR N = COUNTROWS(FactMCScenarios)
       RETURN
           (N*(N+1)) / ((N-1)*(N-2)*(N-3)) *
           SUMX(FactMCScenarios, ((FactMCScenarios[PortfolioPnL] -
   Avg) / Std) ^ 4) -
           3 * (N-1)^2 / ((N-2)*(N-3))
```

A4.10 Signal Quality & Attribution DAX Measures

```
// Signal Precision (True Positives / All Predicted Positives)
   Signal Precision =
       VAR TruePositive =
           COUNTROWS(FILTER(FactSignals, FactSignals[Predicted] =
   "BUY" && FactSignals[Actual] = "UP"))
       VAR AllPositive =
           COUNTROWS(FILTER(FactSignals, FactSignals[Predicted] =
   "BUY"))
       RETURN DIVIDE(TruePositive, AllPositive, 0)

// Signal Recall (True Positives / All Actual Positives)
   Signal Recall =
       VAR TruePositive =
           COUNTROWS(FILTER(FactSignals, FactSignals[Predicted] =
   "BUY" && FactSignals[Actual] = "UP"))
       VAR AllActualPositive =
           COUNTROWS(FILTER(FactSignals, FactSignals[Actual] =
   "UP"))
       RETURN DIVIDE(TruePositive, AllActualPositive, 0)

// F1 Score
   F1 Score =
       VAR P = [Signal Precision]
       VAR R = [Signal Recall]
```

```
        RETURN DIVIDE(2 * P * R, P + R, 0)

// Signal Decay (Hit Rate Over Time Since Signal)
   Signal Decay =
        VAR HoursAfter = SELECTEDVALUE(DimDecay[HoursAfter])
        VAR TotalSig = COUNTROWS(FactSignals)
        VAR StillValid =
            COUNTROWS(
                FILTER(FactSignals,
                    FactSignals[StillCorrectAfterHours] >=
   HoursAfter
                )
            )
        RETURN DIVIDE(StillValid, TotalSig, 0)

// Attribution - Technical vs ML vs RL Contribution
   Technical Contribution =
        SUMX(
            FILTER(FactTrades, FactTrades[SignalSource] =
   "TECHNICAL"),
            FactTrades[PnL]
        )

   ML Contribution =
        SUMX(
            FILTER(FactTrades, FactTrades[SignalSource] = "ML"),
            FactTrades[PnL]
        )

   RL Contribution =
        SUMX(
            FILTER(FactTrades, FactTrades[SignalSource] = "RL"),
            FactTrades[PnL]
        )

// Composite vs Individual Signal Alpha
   Composite Alpha =
        VAR CompositePnL = CALCULATE([Total PnL],
   FactTrades[SignalSource] = "COMPOSITE")
        VAR BestIndividual =
            MAXX(
                {[Technical Contribution], [ML Contribution], [RL
   Contribution]},
                [Value]
            )
        RETURN CompositePnL - BestIndividual
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A4.11 Time Intelligence DAX Measures

```
// Month-to-Date P&L
   MTD PnL =
       CALCULATE(
           [Total PnL],
           DATESMTD(DimDate[Date])
       )

// Quarter-to-Date P&L
   QTD PnL =
       CALCULATE(
           [Total PnL],
           DATESQTD(DimDate[Date])
       )

// Year-to-Date P&L
   YTD PnL =
       CALCULATE(
           [Total PnL],
           DATESYTD(DimDate[Date])
       )

// Prior Month P&L
   Prior Month PnL =
       CALCULATE(
           [Total PnL],
           PREVIOUSMONTH(DimDate[Date])
       )

// Month-over-Month P&L Change
   MoM PnL Change =
       VAR CurrentMonth = [MTD PnL]
       VAR PriorMonth = [Prior Month PnL]
       RETURN DIVIDE(CurrentMonth - PriorMonth, ABS(PriorMonth), 0)

// Rolling 30-Day Sharpe Ratio
   Rolling 30D Sharpe =
       VAR Last30 =
           FILTER(
               ALL(DimDate),
               DimDate[Date] >= MAX(DimDate[Date]) - 30 &&
   DimDate[Date] <= MAX(DimDate[Date])
           )
       VAR DailyPnL =
           ADDCOLUMNS(Last30, "DPnL", CALCULATE([Daily PnL]))
       VAR AvgD = AVERAGEX(DailyPnL, [DPnL])
```

```
        VAR StdD = SQRT(AVERAGEX(DailyPnL, ([DPnL] - AvgD)^2))
        RETURN DIVIDE(AvgD, StdD, 0) * SQRT(252)

// Rolling 90-Day Win Rate
   Rolling 90D Win Rate =
        VAR Last90 =
            FILTER(
                FactTrades,
                FactTrades[TradeDate] >= MAX(DimDate[Date]) - 90
            )
        RETURN DIVIDE(
            COUNTROWS(FILTER(Last90, FactTrades[PnL] > 0)),
            COUNTROWS(Last90), 0)

// Daily P&L Moving Average (20-day)
   PnL MA 20D =
        VAR CurrentDate = MAX(DimDate[Date])
        VAR Last20Days =
            TOPN(20,
                FILTER(ALL(DimDate), DimDate[Date] <= CurrentDate),
                DimDate[Date], DESC
            )
        RETURN AVERAGEX(Last20Days, CALCULATE([Daily PnL]))

// Best Trading Day (This Month)
   Best Day MTD =
        MAXX(
            FILTER(DATESMTD(DimDate[Date]), TRUE()),
            CALCULATE([Daily PnL])
        )

// Worst Trading Day (This Month)
   Worst Day MTD =
        MINX(
            FILTER(DATESMTD(DimDate[Date]), TRUE()),
            CALCULATE([Daily PnL])
        )

// Consecutive Profitable Days
   Consecutive Profit Days =
        VAR DaysTable =
            ADDCOLUMNS(
                SELECTCOLUMNS(DimDate, "D", DimDate[Date]),
                "DPnL", CALCULATE([Daily PnL]),
                "IsProfit", IF(CALCULATE([Daily PnL]) > 0, 1, 0)
            )
```

```
        VAR CurrentDate = MAX(DimDate[Date])
        VAR RecentDays = FILTER(DaysTable, [D] <= CurrentDate)
        RETURN
            COUNTROWS(
                TOPN(
                    999,
                    FILTER(RecentDays, [IsProfit] = 1 && [D] >=
   CALCULATE(MAX([D]), FILTER(RecentDays, [IsProfit] = 0))),
                    [D], DESC
                )
            )
```

A4.12 Position Tracking & Exposure DAX Measures

```
// Current Open Positions Count
   Open Positions = CALCULATE(COUNTROWS(FactTrades),
   FactTrades[IsOpen] = TRUE())

// Net Exposure (Long - Short in USD)
   Net Exposure =
        VAR LongExp =
            SUMX(
                FILTER(FactTrades, FactTrades[IsOpen] = TRUE() &&
   FactTrades[Direction] = "LONG"),
                FactTrades[PositionSize] * FactTrades[CurrentPrice]
            )
        VAR ShortExp =
            SUMX(
                FILTER(FactTrades, FactTrades[IsOpen] = TRUE() &&
   FactTrades[Direction] = "SHORT"),
                FactTrades[PositionSize] * FactTrades[CurrentPrice]
            )
        RETURN LongExp - ShortExp

// Gross Exposure
   Gross Exposure =
        SUMX(
            FILTER(FactTrades, FactTrades[IsOpen] = TRUE()),
            ABS(FactTrades[PositionSize] * FactTrades[CurrentPrice])
        )

// Long/Short Ratio
   Long Short Ratio =
        VAR LongCount = CALCULATE(COUNTROWS(FactTrades),
   FactTrades[IsOpen] = TRUE(), FactTrades[Direction] = "LONG")
        VAR ShortCount = CALCULATE(COUNTROWS(FactTrades),
   FactTrades[IsOpen] = TRUE(), FactTrades[Direction] = "SHORT")
        RETURN DIVIDE(LongCount, ShortCount, 0)
```

```
// Unrealised P&L
   Unrealised PnL =
       SUMX(
           FILTER(FactTrades, FactTrades[IsOpen] = TRUE()),
           IF(FactTrades[Direction] = "LONG",
               (FactTrades[CurrentPrice] - FactTrades[EntryPrice])
   * FactTrades[PositionSize],
               (FactTrades[EntryPrice] - FactTrades[CurrentPrice])
   * FactTrades[PositionSize]
           )
       )

// Total P&L (Realised + Unrealised)
   Total Combined PnL = [Total PnL] + [Unrealised PnL]

// Exposure by Region
   Region Exposure =
       SUMX(
           FILTER(
               FactTrades,
               FactTrades[IsOpen] = TRUE() &&
               RELATED(DimCurrencyPair[Region]) =
   SELECTEDVALUE(DimCurrencyPair[Region])
           ),
           ABS(FactTrades[PositionSize] * FactTrades[CurrentPrice])
       )

// Leverage Ratio
   Leverage Ratio =
       VAR PortfolioValue = 100000  // Initial capital
       RETURN DIVIDE([Gross Exposure], PortfolioValue + [Total
   PnL], 0)

// Margin Utilisation
   Margin Utilisation =
       VAR RequiredMargin = [Gross Exposure] / 50  // 50:1 leverage
       VAR AvailableMargin = 100000 + [Total PnL]
       RETURN DIVIDE(RequiredMargin, AvailableMargin, 0)

// Concentration Risk (Herfindahl Index)
   Concentration HHI =
       VAR PairExposures =
           ADDCOLUMNS(
               VALUES(DimCurrencyPair[PairID]),
               "PairExp",
```

```
                CALCULATE(
                    SUMX(FILTER(FactTrades, FactTrades[IsOpen] =
    TRUE()),
                        ABS(FactTrades[PositionSize] *
    FactTrades[CurrentPrice])
                    )
                )
            VAR TotalExp = SUMX(PairExposures, [PairExp])
            RETURN
                SUMX(PairExposures, (DIVIDE([PairExp], TotalExp, 0)) ^
    2)

// Days in Trade (Average Hold Period)
    Avg Hold Period =
        AVERAGEX(
            FILTER(FactTrades, FactTrades[IsOpen] = FALSE()),
            FactTrades[HoldingDays]
        )

// Trade Turnover Rate
    Turnover Rate =
        VAR TradingDays = DISTINCTCOUNT(FactTrades[DateKey])
        RETURN DIVIDE([Total Trades], TradingDays, 0)
```

A4.13 Trend Detection & Market Regime DAX Measures

```
// Current Trend Direction
    Trend Direction =
        VAR SMAShort = [SMA_N]  // Using 10-period
        VAR SMALong =
            CALCULATE([SMA_N], DimParameter[MA_Period] = 50)
        RETURN
            IF(SMAShort > SMALong, "UPTREND",
                IF(SMAShort < SMALong, "DOWNTREND", "SIDEWAYS"))

// Trend Strength (ADX Approximation)
    Trend Strength =
        VAR Period = 14
        VAR RecentData =
            TOPN(Period,
                FILTER(ALL(FactMarketData),
    FactMarketData[DateTimeKey] <=
    MAX(FactMarketData[DateTimeKey])),
                FactMarketData[DateTimeKey], DESC
            )
        VAR AvgHighLow = AVERAGEX(RecentData,
    ABS(FactMarketData[High] - FactMarketData[Low]))
```

```
        VAR AvgClose = AVERAGEX(RecentData, FactMarketData[Close])
        VAR NormRange = DIVIDE(AvgHighLow, AvgClose, 0) * 100
        RETURN
            IF(NormRange > 1.5, "STRONG",
                IF(NormRange > 0.8, "MODERATE", "WEAK"))

// Volatility Regime
   Volatility Regime =
        VAR Vol20 = [Pair Volatility 20D]
        RETURN
            SWITCH(TRUE(),
                Vol20 < 0.06, "LOW_VOL",
                Vol20 < 0.12, "NORMAL",
                Vol20 < 0.20, "HIGH_VOL",
                "CRISIS"
            )

// Momentum Score
   Momentum Score =
        VAR RSI = [RSI_14]
        VAR MACD_H = [MACD Histogram]
        VAR BB = [BB Percent B]
        VAR RSI_Score = IF(RSI < 30, 1, IF(RSI > 70, -1, 0))
        VAR MACD_Score = IF(MACD_H > 0, 1, -1)
        VAR BB_Score = IF(BB < 0.2, 1, IF(BB > 0.8, -1, 0))
        RETURN (RSI_Score * 0.35 + MACD_Score * 0.35 + BB_Score *
   0.30)

// Signal Consensus (How Many Indicators Agree)
   Signal Consensus =
        VAR RSI_Buy = IF([RSI_14] < 40, 1, 0)
        VAR MACD_Buy = IF([MACD Histogram] > 0, 1, 0)
        VAR BB_Buy = IF([BB Percent B] < 0.3, 1, 0)
        VAR MA_Buy = IF([Trend Direction] = "UPTREND", 1, 0)
        VAR BuyCount = RSI_Buy + MACD_Buy + BB_Buy + MA_Buy
        RETURN
            SWITCH(TRUE(),
                BuyCount >= 3, "STRONG CONSENSUS BUY",
                BuyCount = 2, "MODERATE BUY",
                BuyCount = 1, "WEAK/MIXED",
                "SELL CONSENSUS"
            )

// Mean Reversion Z-Score
   Z Score =
        VAR CurrentPrice = MAX(FactMarketData[Close])
```

```
        VAR SMA = [SMA_N]
        VAR Period = 20
        VAR RecentData =
            TOPN(Period,
                FILTER(ALL(FactMarketData),
    FactMarketData[DateTimeKey] <=
    MAX(FactMarketData[DateTimeKey])),
                FactMarketData[DateTimeKey], DESC
            )
        VAR StdDev = SQRT(AVERAGEX(RecentData,
    (FactMarketData[Close] - SMA) ^ 2))
        RETURN DIVIDE(CurrentPrice - SMA, StdDev, 0)

// Overbought/Oversold Alert
    OB/OS Alert =
        VAR ZS = [Z Score]
        VAR RSI = [RSI_14]
        RETURN
            IF(ZS > 2 && RSI > 70, "OVERBOUGHT - STRONG",
                IF(ZS > 1.5 || RSI > 70, "OVERBOUGHT - MODERATE",
                    IF(ZS < -2 && RSI < 30, "OVERSOLD - STRONG",
                        IF(ZS < -1.5 || RSI < 30, "OVERSOLD -
    MODERATE",
                            "NEUTRAL"))))
```

A4.14 Worked Examples – DAX Measure Validation

Worked Example 1: EUR/USD Signal Generation & P&L Calculation

Given Data (5-day window):

Date	Open	High	Low	Close	SMA_10	SMA_50	RSI	Signal
2024-01-15	1.0920	1.0945	1.0905	1.0938	1.0910	1.0880	62	BUY
2024-01-16	1.0938	1.0960	1.0925	1.0952	1.0918	1.0882	65	BUY
2024-01-17	1.0952	1.0970	1.0930	1.0942	1.0920	1.0885	58	HOLD
2024-01-18	1.0942	1.0950	1.0870	1.0878	1.0912	1.0886	42	HOLD
2024-01-19	1.0878	1.0890	1.0850	1.0865	1.0895	1.0888	35	SELL

DAX Measure Verification:

Total Signals = 5 (COUNTROWS of filtered FactSignals for these dates)

Buy Signals = 2 (Jan 15, Jan 16)

Sell Signals = 1 (Jan 19)

If trade executed: Buy at 1.0938 (Jan 15), Sell at 1.0865 (Jan 19)

PnL = (1.0865 - 1.0938) * 100000 = -$730 (loss of 73 pips)

Spread cost = 2 pips * $10/pip = $20 each way = $40 total

Net PnL = -$730 - $40 = -$770

Win Rate = 0/1 = 0% (this single trade was a loss)
Signal Hit Rate = 0 (the buy signal at RSI 62 did not lead to profitable outcome)

Worked Example 2: Portfolio VaR Calculation
Portfolio: $10M EUR/USD long, $5M USD/JPY long, $3M GBP/USD long
Daily volatilities: EUR/USD 0.55%, USD/JPY 0.70%, GBP/USD 0.65%
Correlations: EUR/USD-USD/JPY: -0.40, EUR/USD-GBP/USD: 0.85, USD/JPY-GBP/USD: -0.30

Individual VaR 95% (1-day):
EUR/USD: $10M * 0.55% * 1.645 = $90,475
USD/JPY: $5M * 0.70% * 1.645 = $57,575
GBP/USD: $3M * 0.65% * 1.645 = $32,078
Undiversified VaR = $90,475 + $57,575 + $32,078 = $180,128

Diversified VaR (using correlation matrix):
$VaR^2 = w' * \Sigma * w$
= [90475, 57575, 32078] * [1, -0.4, 0.85; -0.4, 1, -0.3; 0.85, -0.3, 1] * [90475; 57575; 32078]
= $8.185B + $3.315B + $1.029B + 2*(-2083M) + 2*(2469M) + 2*(-554M) = $9.865B
Diversified VaR = √($9.865B) = $99,324

Diversification benefit = $180,128 - $99,324 = $80,804 (44.9% reduction)
The negative EUR/USD-JPY correlation significantly reduces portfolio risk.

Worked Example 3: Sharpe Ratio Calculation
Given 20 days of daily P&L: [150, -80, 220, -30, 110, -150, 300, -50, 180, -20, 90, -120, 250, -60, 170, -90, 200, -40, 130, -70]
Sum = 1,090 | Mean daily P&L = $54.50
Variance = (1/20) * $\Sigma(x_i - 54.5)^2$ = 18,847.25
Std Dev = √18,847.25 = $137.29
Daily risk-free = 5%/252 = $0.0198 per dollar (assume on $100K: $7.87)
Daily Sharpe = (54.50 - 7.87) / 137.29 = 0.3395
Annualised Sharpe = 0.3395 * √252 = 5.39
This would be verified in the DAX [Sharpe Ratio] measure against the Python calculation.

A4.15 Excel VBA Implementation for Forex Signals

```
Function CalcRSI(rng As Range, period As Integer) As Double
    Dim gains As Double, losses As Double
    Dim i As Integer
    For i = 2 To period + 1
        Dim chg As Double
        chg = rng.Cells(i, 1).Value - rng.Cells(i - 1, 1).Value
        If chg > 0 Then gains = gains + chg Else losses = losses
+ Abs(chg)
    Next i
    Dim avgGain As Double: avgGain = gains / period
    Dim avgLoss As Double: avgLoss = losses / period
    If avgLoss = 0 Then CalcRSI = 100: Exit Function
    CalcRSI = 100 - (100 / (1 + avgGain / avgLoss))
End Function

Function SignalScore(rsi As Double, macdHist As Double, bbPctB
As Double) As String
    Dim score As Double: score = 0
    If rsi < 30 Then score = score + 1
    If rsi > 70 Then score = score - 1
    If macdHist > 0 Then score = score + 1 Else score = score -
1
    If bbPctB < 0.2 Then score = score + 1
    If bbPctB > 0.8 Then score = score - 1
    If score >= 2 Then SignalScore = "BUY"
    ElseIf score <= -2 Then SignalScore = "SELL"
    Else SignalScore = "HOLD"
    End If
End Function
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Excel Formulas for FX Signal Generation

Formula	Purpose	Cell Reference
=AVERAGE(D2:D11)	10-Period Short MA	E2
=AVERAGE(D2:D51)	50-Period Long MA	F2
=IF(AND(E2>F2,E1<=F1),"BUY",IF(AND(E2<F2,E1>=F1),"SELL","HOLD"))	Crossover Signal	G2
=STDEV(D2:D21)*SQRT(252)	20-day Annualised Volatility	H2
=COUNTIF(G2:G251,"BUY")/COUNTA(G2:G251)	Buy Signal Frequency	I2
=SUMPRODUCT((G2:G251="BUY")*(K2:K251>0))/COUNTIF(G2:G251,"BUY")	Buy Signal Hit Rate	J2

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A5: DATA REQUIREMENTS & STRUCTURE

A5.1 Forex Price Data Structure

Field	Type	Description	Example
DateTime	Datetime	Candle timestamp in UTC	2024-01-15 14:30:00
PairID	Integer	Foreign key to DimCurrencyPair	1 (EUR/USD)
Open	Decimal(10,5)	Opening price of candle	1.08750
High	Decimal(10,5)	Highest price in candle	1.08820
Low	Decimal(10,5)	Lowest price in candle	1.08690
Close	Decimal(10,5)	Closing price of candle	1.08780
Volume	Integer	Tick volume	4532
Spread	Decimal(6,1)	Bid-ask spread in pips	1.2

A5.2 Trade Log Structure

Field	Type	Description
TradeID	Integer PK	Unique trade identifier
PairID	Integer FK	Currency pair traded
StrategyID	Integer FK	Strategy that generated signal
SignalType	String	BUY / SELL / STRONG_BUY / STRONG_SELL
EntryPrice	Decimal	Trade entry price
ExitPrice	Decimal	Trade exit price
EntryDateTime	Datetime	Entry timestamp
ExitDateTime	Datetime	Exit timestamp
PositionSize	Integer	Position in base currency units
PnL	Decimal	Realised profit/loss in USD
PnL_Pips	Decimal	P&L in pips
HoldingDays	Decimal	Duration of trade in days
Commission	Decimal	Transaction cost
SlippageEstimate	Decimal	Estimated slippage cost

SECTION A6: SCENARIO ANALYSIS FRAMEWORK

A6.1 Central Bank Divergence Scenarios

Standard scenarios for testing FX signal robustness across different monetary policy regimes:

Scenario	Fed	ECB	BOJ	RBI	Impact
Hawkish Convergence	+50bps	+50bps	+25bps	+25bps	USD weakens, JPY strengthens
Fed Pivot	-75bps	Hold	Hold	Hold	Broad USD sell-off, EM rally
BOJ Shock Tightening	Hold	Hold	+100bs	Hold	JPY massive rally, carry unwind
RBI Surprise Cut	Hold	Hold	Hold	-50bps	INR weakens, equity rally
Global Risk-Off	+25bps	+25bps	Hold	+25bps	USD/JPY/CHF safe haven bid
EM Crisis Contagion	Hold	Hold	Hold	Multiple	INR under pressure, capital flight

A6.2 Volatility Regime Scenarios

FX signal effectiveness varies dramatically by volatility regime. The following scenarios test signal adaptation:

- Low Volatility (VIX < 15): Mean-reversion signals dominate; carry trades perform well; signal frequency low
- Normal Volatility (VIX 15-25): Trend-following and mean-reversion both viable; balanced signal generation
- High Volatility (VIX 25-35): Trend-following signals dominate; stop-losses widen; position sizes reduce
- Crisis Volatility (VIX > 35): All signals unreliable; risk management overrides; cash preservation mode

Python Implementation – Volatility Regime Detection:

```
def detect_regime(df, lookback=60):
    df['RealizedVol'] =
df['Close'].pct_change().rolling(lookback).std() * np.sqrt(252)
    conditions = [
        df['RealizedVol'] < 0.06,
        (df['RealizedVol'] >= 0.06) & (df['RealizedVol'] <
0.12),
        (df['RealizedVol'] >= 0.12) & (df['RealizedVol'] <
0.20),
        df['RealizedVol'] >= 0.20
    ]
```

```
    regimes = ['LOW_VOL', 'NORMAL', 'HIGH_VOL', 'CRISIS']
    df['Regime'] = np.select(conditions, regimes,
default='NORMAL')
    return df
```

A6.3 Multi-Pair Correlation Breakdown Scenario

During normal markets, EUR/USD and GBP/USD correlation is ~0.85. During Brexit-style events, this correlation can drop to 0.2 or even turn negative. The signal engine must detect and adapt to correlation regime changes.

SECTION A7: MONTE CARLO SIMULATION FOR FX PORTFOLIOS

A7.1 Geometric Brownian Motion for FX Rates

```
class FXMonteCarloEngine:
    def __init__(self, pairs, corr_matrix, n_sims=10000,
horizon=252):
        self.pairs = pairs
        self.corr = corr_matrix
        self.n_sims = n_sims
        self.horizon = horizon

    def generate_paths(self, spot_prices, vols, drifts):
        n = len(self.pairs)
        L = np.linalg.cholesky(self.corr)
        dt = 1 / 252
        paths = np.zeros((self.n_sims, self.horizon, n))
        for i in range(n):
            paths[:, 0, i] = spot_prices[i]
        for t in range(1, self.horizon):
            Z = np.random.standard_normal((self.n_sims, n))
            corr_Z = Z @ L.T
            for i in range(n):
                paths[:, t, i] = paths[:, t-1, i] * np.exp(
                    (drifts[i] - 0.5 * vols[i]**2) * dt +
vols[i] * np.sqrt(dt) * corr_Z[:, i])
        return paths

    def compute_portfolio_var(self, paths, positions,
conf=0.95):
        final_values = np.sum(paths[:, -1, :] * positions,
axis=1)
        initial_value = np.sum(paths[:, 0, :] * positions)
        pnl = final_values - initial_value
        var = np.percentile(pnl, (1 - conf) * 100)
        cvar = np.mean(pnl[pnl <= var])
        return var, cvar, pnl
```

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

SECTION B1: FX SIGNAL LAB – SIMULATION OVERVIEW

B1.1 Business Rationale and Problem Statement

Despite the FX market being the world's largest, institutional-quality signal generation training remains virtually inaccessible to students. Trading desks recruit analysts who can combine macro analysis, technical indicators, and quantitative models – yet no simulation platform teaches this integrated approach.

FX Signal Lab fills this gap by providing a gamified environment where trainees build, test, and compete with AI-augmented trading signal systems across realistic market scenarios.

The Training Gap this Platform Solves

- University finance courses teach FX theory but not signal generation methodology
- Existing demo accounts provide execution practice but zero signal development training
- No platform teaches integration of RL agents with traditional technical analysis
- Power BI dashboard design for FX trading desks is untaught in formal education

Target Audience

- Tier 1: MBA/MSF students specialising in trading or quantitative finance
- Tier 2: Undergraduate CS/Engineering students interested in algorithmic trading
- Tier 3: Early-career FX analysts at banks and hedge funds
- Tier 4: Proprietary trading firms seeking standardised assessment tools

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

B1.2 Simulation Architecture

Component	Description	Technology
Signal Engine	Generates signals from multiple sources (TA, ML, RL)	Python, TensorFlow, stable-baselines3
Market Simulator	Replays historical FX data with configurable speed	Python, WebSocket, Redis
DAX Dashboard	Real-time Power BI dashboards with 120+ measures	Power BI, DAX Studio
Scoring Engine	Evaluates signal quality, risk management, P&L	Python, PostgreSQL
Leaderboard	Competitive ranking across cohorts	React, Node.js, MongoDB
Scenario Generator	Injects central bank events, flash crashes, etc.	Python, Monte Carlo engine

B1.3 Game Modes

Campaign Mode (Primary – 8 Levels):

- Level 1 – The Trend Follower: Single pair (EUR/USD), generate MA crossover signals. Score based on signal accuracy.
- Level 2 – The Oscillator: Add RSI/MACD/Bollinger. Combine signals into composite scores. Beat benchmark by 20%.
- Level 3 – The Carry Trader: Identify and exploit interest rate differentials across 5 pairs. Risk-adjusted carry returns.
- Level 4 – The ML Engineer: Train Random Forest and XGBoost classifiers. Achieve >55% accuracy on out-of-sample data.
- Level 5 – The Deep Learner: Build LSTM model for sequential signals. Beat tree-based models on Sharpe Ratio.
- Level 6 – The RL Agent: Train PPO agent in custom FX environment. Agent must generate positive P&L in live simulation.
- Level 7 – The Risk Manager: Build VaR/CVaR framework. All trades must pass risk checks. Survive 3 stress scenarios.
- Level 8 – The Portfolio Master: Multi-pair portfolio with correlation management. Beat Sharpe 1.5 across all regimes.

SECTION B2: SCORING SYSTEM AND GAME MECHANICS

B2.1 1000-Point Scoring Framework

Category	Points	Criteria
Signal Accuracy	250	Hit rate, precision, recall, F1 score across all signal types
Risk Management	200	VaR compliance, drawdown limits, position sizing, stop-loss discipline
P&L Performance	200	Absolute return, Sharpe ratio, profit factor, expectancy
ML Model Quality	150	Out-of-sample accuracy, feature importance, cross-validation stability
DAX Dashboard	100	Completeness, interactivity, visual design, measure accuracy
Speed & Reporting	100	Time to signal, documentation quality, presentation clarity

B2.2 Progression System (8 Levels)

Level	Title	Points Required	Unlock
1	Signal Novice	0	Basic MA crossover tools
2	Indicator Analyst	150	Advanced oscillators and composites
3	Carry Strategist	300	Multi-pair carry analysis
4	ML Signal Scientist	450	Random Forest, XGBoost classifiers
5	Deep Signal Architect	600	LSTM, attention-based models
6	RL Trading Agent	750	PPO, DQN, A2C environments
7	Risk Sentinel	875	Full VaR/CVaR framework, stress testing
8	FX Signal Master	950	Multi-pair portfolio, all scenarios

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

B2.3 Sample Scenario: The Boj Shock

Scenario: Bank of Japan unexpectedly raises rates by 100bps (reminiscent of July 2024 carry unwind).

- USD/JPY drops from 155.00 to 142.00 in 48 hours
- AUD/JPY and NZD/JPY crash as carry trades unwind globally
- EUR/USD initially rallies then reverses as risk-off hits equities
- Signal engine must: detect regime change, flip from carry to safe-haven mode, resize positions

Expected Student Actions: Activate crisis mode, cut all JPY-funded carry positions, switch to risk-off signal weights, monitor cross-asset correlations, recalibrate VaR with stressed parameters.

B2.4 Additional Sample Scenarios

Scenario 2: The Swiss Franc Redux

The Central bank unexpectedly removes a currency peg. USD/CHF gaps 15% in seconds. The signal engine must detect gaps, suspend trading in affected pairs, recalculate all open position P&L, alert risk limits breached.

- Player must: Identify peg-removal risk, limit single-pair exposure to 15% max, have emergency stop protocols
- Scoring: 50 points for pre-positioning limits, 30 points for speed of response, 20 points for P&L recovery

Scenario 3: The NFP Surprise

US Non-Farm Payrolls come in at +350K vs +180K expected. Dollar surges across all pairs. EUR/USD drops 80 pips in 5 minutes. The signal engine must handle high-volatility event trading.

- Player must: Reduce position sizes pre-NFP, widen stop-losses for event risk, avoid trading first 5 minutes
- Scoring: 40 points for pre-event risk reduction, 30 points for post-event signal accuracy, 30 points for execution quality

Scenario 4: The Correlation Breakdown

EUR/USD and GBP/USD historically correlated at 0.85 suddenly decouple due to the UK-specific political crisis. Pairs move in opposite directions for 5 consecutive days. Portfolio diversification assumptions break down.

- Player must: Detect correlation shift, recalculate portfolio VaR with updated correlation matrix, rebalance hedges
- Scoring: 40 points for correlation detection speed, 30 points for VaR recalculation, 30 points for hedge adjustment

Scenario 5: The Flash Crash

GBP/USD drops 600 pips in 2 minutes during thin Asian session liquidity. Spreads widen to 50 pips. Stop-losses trigger at prices far from set levels due to gapping. Signal engine must handle extreme slippage.

- Player must: Implement session-aware position sizing, use limit orders not market orders, have liquidity scoring
- Scoring: 30 points for session-aware sizing, 30 points for order type discipline, 40 points for recovery strategy

B2.5 Business Model And Monetisation

Revenue Stream	Price	Target	Annual Revenue
Student Subscription	$15/month	50,000 users Year 3	$9,000,000
University Site License	$5,000/year	200 universities	$1,000,000
Prop Firm Assessment	$200/assessment	10,000 assessments	$2,000,000
Corporate Training	$50,000/cohort	50 cohorts	$2,500,000
API Data Feed	$500/month	500 subscribers	$3,000,000

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

SECTION C1: USD/JPY CARRY TRADE UNWIND – JULY-AUGUST 2024

C1.1 Background

In July 2024, the Bank of Japan surprised markets with a rate hike, triggering the most violent carry trade unwind since 1998. USD/JPY fell from approximately 162 to 142 within weeks, causing cascading losses across global carry portfolios. The Nikkei experienced its largest single-day percentage drop since 1987.

This event exposed the fundamental fragility of crowded carry trades and the failure of standard signal systems to detect regime shifts in real-time.

C1.2 Signal System Analysis

Pre-Shock Signal State: Most MA crossover systems were long USD/JPY (10-day EMA above the 50-day EMA since January 2024). RSI at 68 – close to overbought but not triggering sell signals. MACD positive but histogram declining.

The signal system should have detected: declining MACD histogram, rising implied volatility (USD/JPY 1M vol jumped from 8% to 15%), and BOJ forward guidance shift in preceding meetings.

C1.3 Quantitative Analysis

Portfolio Impact: A standard carry portfolio long AUD/JPY ($10M), NZD/JPY ($5M), and USD/JPY ($15M) would have lost approximately $2.8M in 5 trading days – a drawdown of approximately 9.3%.

VaR Failure: 95% parametric VaR using 60-day historical volatility was approximately $350K. Actual loss was 8x VaR, demonstrating the tail risk of carry trades.

C1.4 Dax Measures For Carry Trade Monitoring

```
Carry Position Exposure =
    SUMX(
          FILTER(FactTrades, FactTrades[TradeType] = "CARRY" &&
FactTrades[IsOpen] = TRUE()),
        FactTrades[PositionSize] * FactTrades[CurrentPrice]
    )

Carry Unwind Alert =
          VAR JPYVol = CALCULATE([Pair Volatility 20D],
DimCurrencyPair[PairName] = "USD/JPY")
         VAR JPYVolMA = CALCULATE([Pair Volatility 60D],
DimCurrencyPair[PairName] = "USD/JPY")
    VAR VolRatio = DIVIDE(JPYVol, JPYVolMA, 1)
     RETURN IF(VolRatio > 1.5, "HIGH ALERT", IF(VolRatio > 1.2,
"WARNING", "NORMAL"))

Carry-at-Risk =
    [Carry Position Exposure] * [Pair Volatility 20D] * 1.645 /
SQRT(252)
```

SECTION C2: GBP FLASH CRASH – OCTOBER 2016

C2.1 Background

On 7 October 2016, GBP/USD fell approximately 6% in 2 minutes during Asian trading hours, from 1.2600 to 1.1841. The flash crash was attributed to algorithmic trading amplifying thin liquidity conditions during a typically quiet trading session.

C2.2 Signal Implications

Trend-following signals correctly captured the post-Brexit downtrend but were unprepared for the velocity of the move. Stop-loss orders were triggered at prices far below intended levels due to market gapping.

Key lesson: Signals must incorporate liquidity and session awareness. The DAX dashboard should include:

```
Liquidity Score =
    VAR CurrentHour = HOUR(NOW())
    VAR AvgVolume =
        CALCULATE(
            AVERAGEX(FactMarketData, FactMarketData[Volume]),
            DimTime[Hour] = CurrentHour
        )
            VAR   OverallAvg   =   AVERAGEX(ALL(FactMarketData),
FactMarketData[Volume])
    RETURN DIVIDE(AvgVolume, OverallAvg, 0)

Session Risk Multiplier =
    SWITCH(
        SELECTEDVALUE(DimTime[Session]),
        "London", 1.0,
        "NY", 1.0,
        "London-NY Overlap", 0.8,
        "Tokyo", 1.3,
        "Sydney", 1.5,
        1.2
    )
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C3: USD/INR RANGE TRADE – RBI INTERVENTION 2023-2024

C3.1 Background

Throughout 2023-2024, the RBI maintained a tight range in USD/INR between 82.50 and 83.50, intervening heavily on both sides. This created a unique mean-reversion opportunity where standard trend-following signals failed.

C3.2 Signal Adaptation

Signal systems designed for G10 pairs with trending behaviour produced false breakout signals when applied to USD/INR. The solution: detect range-bound regimes and switch to mean-reversion signals automatically.

```
Regime Detection DAX =
    VAR Range20D =
                                MAXX(TOPN(20,    FactMarketData,
FactMarketData[DateTimeKey], DESC), FactMarketData[High])
                            -   MINX(TOPN(20,    FactMarketData,
FactMarketData[DateTimeKey], DESC), FactMarketData[Low])
    VAR AvgRange = [ATR_14] * 20
    VAR RangeRatio = DIVIDE(Range20D, AvgRange, 1)
     RETURN IF(RangeRatio < 0.6, "RANGE_BOUND", IF(RangeRatio >
1.4, "TRENDING", "TRANSITIONING"))
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C4: EUR/CHF FLOOR REMOVAL – JANUARY 2015

C4.1 Background

On 15 January 2015, the Swiss National Bank unexpectedly removed its EUR/CHF floor at 1.2000, causing EUR/CHF to plunge from 1.2000 to 0.8500 in minutes – a 30% move. Multiple FX brokers went bankrupt and several hedge funds suffered terminal losses.

C4.2 Risk Management Lessons For Signal Systems

This black swan event demonstrated that no signal system can predict central bank policy shocks. The critical insight: risk management (position sizing, diversification, maximum loss limits) must be independent of signal quality.

```
Black Swan Protection =
    VAR MaxSinglePairExposure = 0.15  // 15% of portfolio
    VAR CurrentExposure =
        DIVIDE(
            CALCULATE(SUM(FactTrades[PositionSize]),
                FactTrades[PairID] =
SELECTEDVALUE(DimCurrencyPair[PairID]),
                FactTrades[IsOpen] = TRUE()),
            [Total Portfolio Value], 0)
    RETURN IF(CurrentExposure > MaxSinglePairExposure, "BREACH",
"OK")
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

SECTION D1: OVERVIEW

This project follows an AI-accelerated agile methodology where students leverage LLM tools to compress what would traditionally be a 3-month project into 15 intensive days. The 80/20 rule applies: 80% AI-generated code with 20% custom domain integration and validation.

SECTION D2: DAY-BY-DAY BREAKDOWN

Day 1 – Domain Orientation & Environment Setup:

- Read this entire training guide (Parts A through F) thoroughly
- Set up Python environment: conda create -n fxsignal python=3.10
- Install all required libraries: pandas, numpy, scikit-learn, xgboost, tensorflow, stable-baselines3, shap, matplotlib, seaborn
- Download sample FX data from provided CSV or Yahoo Finance API
- Set up Git repository (PRIVATE) and initialise project structure
- Install Power BI Desktop and DAX Studio
- Deliverable: Environment confirmation screenshot, initial project README

Day 2 – Fx Data Ingestion & Exploration:

- Load OHLCV data for 5 currency pairs: EUR/USD, USD/JPY, GBP/USD, AUD/USD, USD/INR
- Compute basic statistics: mean, std, skewness, kurtosis of returns
- Visualise price series, return distributions, and volatility clustering
- Implement data quality checks: missing values, outliers, weekend gaps
- Calculate rolling correlations between pairs
- Deliverable: Jupyter notebook with EDA, data quality report

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Day 3 – Technical Indicators & Signal Generation:

- Implement all technical indicators: SMA, EMA, RSI, MACD, Bollinger Bands, Ichimoku, ATR
- Build CompositeSignalEngine with weighted voting mechanism
- Generate buy/sell signals for all 5 pairs across 2 years of data
- Visualise signals on price charts with entry/exit markers
- Calculate basic signal statistics: frequency, distribution, clustering
- Deliverable: Signal generation module, signal visualisation notebook

Day 4 – Backtesting Framework:

- Build comprehensive backtester with transaction costs, slippage, and spread
- Implement walk-forward optimisation for MA parameters
- Calculate performance metrics: Win Rate, Sharpe, Profit Factor, Max Drawdown
- Compare MA-only vs Composite signals across all pairs
- Generate equity curve and drawdown plots
- Deliverable: Backtesting module, performance comparison report

Day 5 – Machine Learning Signal Models:

- Engineer 30+ features using feature engineering pipeline
- Train Random Forest classifier with TimeSeriesSplit cross-validation
- Train XGBoost classifier with early stopping
- Train LSTM model for sequential signal prediction
- Compare model performance: accuracy, precision, recall, F1
- Generate SHAP explanations for best model
- Deliverable: ML training notebook, model comparison table, SHAP plots

Day 6 – Reinforcement Learning Agent:

- Implement custom ForexTradingEnv (gym.Env subclass)
- Train PPO agent with 100K timesteps on training data
- Train DQN agent for comparison
- Evaluate RL agents on held-out test data
- Compare RL signals vs ML signals vs Technical signals
- Deliverable: RL training scripts, agent evaluation report

Day 7 – Case Study Research & Analysis:

- Research and document 4 FX case studies (see Part C)
- Quantify P&L impact of each event on a model portfolio
- Analyse how different signal systems would have performed
- Propose signal adaptations for each scenario type
- Deliverable: Case study report (minimum 8 pages)

Day 8 – Risk Management Framework:

- Implement parametric VaR (95% and 99%) calculation
- Implement historical simulation VaR
- Build Monte Carlo VaR with correlated multi-pair simulation
- Calculate CVaR/Expected Shortfall
- Implement position sizing using Kelly Criterion and Half-Kelly
- Deliverable: Risk management module, VaR comparison report
-

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Day 9 – R Implementation:

- Replicate core signal generation in R using tidyverse
- Train Random Forest and XGBoost models in R using caret
- Implement Monte Carlo simulation in R
- Create R Shiny dashboard with basic signal monitoring
- Generate publication-quality ggplot2 visualisations
- Deliverable: R scripts, Shiny app prototype

Day 10 – Power Bi Data Model & Core Dax:

- Design and implement star schema in Power BI
- Import all fact and dimension tables
- Create relationships and validate data model
- Implement first 40 DAX measures (A4.1 through A4.4)
- Build first dashboard page: Signal Overview
- Deliverable: Power BI file with data model, 40 DAX measures

Day 11 – Advanced Dax & Dashboard Pages:

- Implement remaining 80+ DAX measures (A4.5 through A4.10)
- Build dashboard pages: Risk Management, Pair Analysis, Session Heatmap
- Create Monte Carlo results page with VaR visualisation
- Add Strategy Comparison page with dynamic ranking
- Implement signal decay analysis visualisation
- Deliverable: Power BI file with 120+ DAX measures, 6 dashboard pages

Day 12 – Dax Studio Optimisation & Validation:

- Export all DAX measures to DAX Studio for performance testing
- Optimise slow-running measures using SUMMARIZE vs ADDCOLUMNS patterns
- Validate DAX calculations against Python/R results
- Cross-check VaR, Sharpe, and drawdown calculations
- Document all measures with descriptions and business logic
- Deliverable: DAX validation report, optimised measures

Day 13 – Gamification Concept Design:

- Design complete 8-level campaign progression (Part B)
- Define scoring rubrics for each level
- Create 5 detailed scenario scripts with expected outcomes
- Design leaderboard and achievement badge system
- Draft business model and 5-year revenue projections
- Deliverable: Gamification design document

Day 14 – Validation, Polish & Integration:

- End-to-end validation of signal pipeline: data ingestion through P&L
- Cross-validate all metrics between Python, R, Power BI, and Excel
- Polish all visualisations and dashboards
- Write final report sections and executive summary
- Peer review of code and documentation
- Deliverable: Validated, polished, integrated deliverables

Day 15 – Final Submission And Handover:

- Transfer GitHub repository ownership to @ZethetaIntern
- Upload all deliverables to shared drive
- Submit final report, code, dashboards, and presentation
- Record 10-minute demonstration video
- Complete exit documentation and knowledge transfer
- Deliverable: All 6 deliverables submitted and handed over

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION D3: MANDATORY DELIVERABLES CHECKLIST

Deliverable 1: Main Report

- Minimum 30 pages covering theory, methodology, results, case studies
- All formulas must be shown with derivations
- Signal comparison tables with statistical significance tests
- Backtesting results with equity curves and drawdown analysis

Deliverable 2: Python Codebase

- Signal generation module with all indicators
- ML training pipeline with cross-validation
- RL training environment and agents (PPO, DQN)
- Monte Carlo simulation engine
- Backtesting framework with transaction costs

Deliverable 3: R Codebase

- Signal generation in R with tidyverse
- ML models using caret and XGBoost
- Monte Carlo simulation and VaR in R
- R Shiny dashboard prototype

Deliverable 4: Power Bi Dashboard

- Star schema data model with 7+ tables
- Minimum 120 DAX measures
- 6+ dashboard pages with interactivity
- DAX Studio validation report

Deliverable 5: Excel Validation Workbook

- Manual signal calculation for 3 pairs (first 100 rows)
- VBA functions for RSI, Signal Score, Backtest
- Cross-validation against Python results

Deliverable 6: Final Presentation

- 15-slide presentation covering methodology, results, gamification concept
- 10-minute recorded demonstration video

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART E: TOOLS & PLATFORMS GUIDE

SECTION E1: PYTHON ENVIRONMENT

E1.1 Required Libraries

```
pip install pandas numpy scipy scikit-learn xgboost tensorflow
pip install stable-baselines3 gym shap matplotlib seaborn plotly
pip install yfinance ta-lib ccxt websocket-client
```

Essential Python Packages

Package	Purpose	Version
pandas	Data manipulation and time series	>=2.0
numpy	Numerical computing	>=1.24
scikit-learn	ML classifiers and cross-validation	>=1.3
xgboost	Gradient boosting classifier	>=2.0
tensorflow	LSTM neural networks	>=2.14
stable-baselines3	Reinforcement learning agents (PPO, DQN, A2C)	>=2.1
shap	Model explainability	>=0.43
matplotlib/seaborn	Visualisation	>=3.7/>=0.12
yfinance	FX data download	>=0.2
gym	RL environment framework	>=0.26

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION E2: R ENVIRONMENT

E2.1 Required Packages

```
install.packages(c('tidyverse','caret','xgboost','randomForest',
'keras',


'shiny','plotly','quantmod','TTR','PerformanceAnalytics','reticu
late'))
```

SECTION E3: POWER BI AND DAX

Power BI Desktop: Free download from Microsoft (powerbi.microsoft.com)

- Version: Latest monthly release
- Data connectivity: CSV, Excel, PostgreSQL, Python scripts
- Custom visuals: Install Deneb for Vega-Lite custom charts

DAX Studio: Free download (daxstudio.org)

- Use for: Query performance tuning, measure debugging, server timing analysis
- Key features: Query plan visualisation, VertiPaq Analyzer, benchmark mode
- Best practice: Test every DAX measure in DAX Studio before deploying to Power BI

Tabular Editor: Free/paid versions (tabulareditor.com)

- Use for: Bulk measure creation, Best Practice Analyzer, documentation generation
- Key features: Script-based measure creation, dependency analysis, perspective management

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION E4: DATA SOURCES

Source	Data	Access	Cost
Yahoo Finance (yfinance)	Daily/Hourly FX rates	Python API	Free
Alpha Vantage	Intraday FX data	REST API	Free tier available
OANDA	Historical tick data	REST API	Free demo account
Dukascopy	Tick-by-tick historical	Java API	Free
RBI Database	USD/INR reference rate	Web scraping	Free
CCIL (India)	G-Sec and forex data	Registered access	Free

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION E5: AI RESEARCH TOOLS

Permitted AI Tools and Usage Guidelines:

- Claude.ai: Primary code generation and documentation assistant
- ChatGPT: Secondary code generation and debugging
- Cursor: AI-integrated IDE for code completion and refactoring
- GitHub Copilot: In-editor code suggestions

Mandatory Validation Requirements:

- ALL AI-generated code must be tested with real data before submission
- ALL DAX measures must be validated in DAX Studio against Python calculations
- ALL ML models must include cross-validation results, not just single-split accuracy
- ALL financial calculations must be cross-checked with manual Excel calculations

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

END OF PROJECT DOCUMENT